

Project Thesis

Data Science

Towards AI-powered Crypto Trading

Authors	Mikail Memis Trojan Veseli
Primary Supervisor	Jasmina Bogojeska
Secondary Supervisor	Christian Badertscher
Date	20.12.24

Declaration of Originality

I hereby declare that I have written this thesis independently or together with the listed group members.

I have only used the sources and aids (including websites and generative AI tools) specified in the text or appendix. I am responsible for the quality of the text and the selection of all content and have ensured that information and arguments are substantiated or supported by appropriate scientific sources. Generative AI tools have been summarized by name and purpose.

Any misconduct will be dealt with according to paragraphs 39 and 40 of the General Academic Regulations for Bachelor's and Master's Degree courses at the Zurich University of Applied Sciences (Rahmenprüfungsordnung ZHAW (RPO)) and subject to the provisions for disciplinary action stipulated in the University regulations.

Place, Date: Zurich, 20.12.2024 **Student names:** Mikail Memis, Trojan Veseli

Acknowledgements

We sincerely thank Prof. Dr. Jasmina Bogojeska for her expertise in artificial intelligence and Dr. Christian Badertscher for his insights into cryptocurrency markets. Their guidance and support were invaluable to the successful completion of this project.

Abstract

Bitcoin’s rapid growth and extreme volatility make it a compelling yet challenging asset for traders. This study applies a **Double Deep Q-Network (DDQN)** to develop reinforcement learning-based trading strategies tailored to Bitcoin markets. The agent uses historical price data and **technical indicators** derived from OHLCV data to make sequential trading decisions.

Two trading strategies are analyzed: a **long-only** strategy, which allows holding and selling assets, and a **short-selling** strategy, which adds the flexibility of profiting from price declines. Both are benchmarked against the **Buy and Hold** approach, evaluating **cumulative return**, **average daily returns**, **standard deviation**, and the **Sharpe ratio** across multiple training cutoff dates.

The results show that the short-selling strategy (cutoff: **July 2021**) achieved the highest **cumulative return** of **86,199.60** and the best **Sharpe ratio** of **0.9168**. However, the long-only strategy demonstrated consistent performance during bullish trends, aligning better with Bitcoin’s historical upward trajectory.

This study highlights the strengths and limitations of reinforcement learning for volatile markets and identifies the potential of incorporating real-time **blockchain metrics** and refined reward functions for further improvements.

Keywords: Deep Reinforcement Learning, Double Deep Q-Network, Bitcoin Trading, Technical Indicators, Long-Only Strategy, Short-Selling Strategy

Contents

1	Introduction	1
1.1	Structure of the Thesis	2
2	Related Work	3
3	Data	5
4	Methods	10
4.1	Overview of the Proposed Method: Double Deep Q-Network (DDQN)	10
4.1.1	State	11
4.1.2	Action	11
4.1.3	Reward Function	13
4.1.4	DQN Loss	13
4.1.5	Experience Replay	14
4.1.6	Target Network Update	15
4.1.7	Exploration-Exploitation Balance	15
4.1.8	Summary of Training Procedure	16
5	Results	18
5.1	Hyperparameter Overview	20
5.2	Training Performance	21
5.3	Long-Only Variant	22
5.4	Short-Selling Variant	25
5.5	Tuned Long-Only Variant	28
5.5.1	Performance Comparison	28
5.5.2	Analysis of Tuned Performance	29
5.5.3	Concluding Remarks on Tuned Parameters	30
5.6	Comparison and Summary	30
5.6.1	Summary of Findings	31
6	Discussion and Conclusion	33
6.1	Discussion	33
6.1.1	Long-Only Variant: Interpretation of Results	33
6.1.2	Tuned Long-Only Variant	34

6.1.3	Short-Selling Variant: Interpretation of Results	34
6.1.4	Discussion of Methods and Limitations	35
6.1.5	The Role of Optimized Data Sources	36
6.1.6	Future Work: Algorithmic Improvements	36
6.2	Conclusion	37
6.3	Appendix	40
6.3.1	Artificial Intelligence Declaration	40

1 Introduction

Bitcoin and other cryptocurrencies have emerged as one of the most discussed financial assets of the 21st century, attracting investors, traders, and researchers alike. Their rapid growth, extreme price volatility, and 24/7 trading availability make them a highly dynamic yet unpredictable market. Successfully navigating such markets requires not only precise timing but also robust strategies capable of adapting to sudden changes in price trends. In recent years, the combination of artificial intelligence (AI) and financial trading has opened new opportunities to automate and optimize decision-making in these volatile environments.

The emergence of artificial intelligence (AI) has catalyzed significant transformations across various sectors, with financial markets being a particularly compelling area for its application. Within this domain, deep reinforcement learning (DRL) has shown considerable promise for tackling sequential decision-making problems inherent in trading. Unlike conventional machine learning techniques that primarily focus on prediction or classification, reinforcement learning agents learn optimal strategies by interacting directly with an environment and receiving feedback through rewards or penalties based on their actions [1].

Cryptocurrency markets, characterized by their 24/7 operation, high-frequency data generation, and significant volatility, present a fertile ground for the application of DRL. These markets offer numerous trading opportunities due to their unique characteristics, including the inter- and intra-asset dependencies highlighted by Zhang et al. [3]. The decentralized nature of cryptocurrencies ensures a more level playing field compared to traditional financial markets, with prices directly reflecting market supply and demand, making them particularly suitable for algorithmic trading approaches. Furthermore, the relative efficiency in price discovery, combined with high liquidity and low transaction costs, provides an ideal environment for DRL-based trading systems.

However, the application of reinforcement learning to real-world financial trading introduces unique challenges, including high-dimensional continuous state and action spaces, non-stationarity of data, and the need for safe exploration strategies to minimize financial risk [11]. This paper introduces a novel approach using a Double Deep Q-Network (DDQN) architecture for Bitcoin trading. Our method builds upon the foundational work of van Hasselt et al. [18], who introduced DDQN to address the overestimation bias in traditional DQN approaches. We extend their framework to the cryptocurrency domain, specifically focusing on Bitcoin (BTC).

1.1 Structure of the Thesis

Section 2 reviews the relevant literature on DRL applications in financial markets. Section 3 describes the dataset and the technical indicators used. Section 4 details the proposed DDQN methodology, including state representation, action space, reward function, and training procedure. Section 5 presents the experimental results, comparing the performance of long-only and short-selling strategies against a Buy and Hold benchmark. Section 6 discusses the findings, limitations, and potential improvements, and also concludes the paper and outlines future research directions.

2 Related Work

The application of reinforcement learning (RL) in financial trading has gained increasing attention in recent years, driven by advancements in deep learning and the availability of high-frequency financial data. Early work in this area focused on traditional RL algorithms, such as Q-learning and SARSA, applied to simplified trading environments [12]. These studies demonstrated the potential of RL agents to learn profitable trading strategies, but were often limited by the curse of dimensionality and the computational challenges of scaling to high-dimensional state and action spaces.

The advent of deep reinforcement learning (DRL), combining deep neural networks with RL algorithms, has significantly expanded the capabilities of trading agents. One of the seminal works in DRL, the Deep Q-Network (DQN) algorithm introduced by Mnih et al. [7], showcased the ability of deep neural networks to approximate the optimal action-value function in complex environments. This breakthrough paved the way for the development of more sophisticated DRL-based trading systems.

In the context of cryptocurrency trading, several studies have explored the application of DRL algorithms. For instance, Jiang et al. [13] proposed a deep reinforcement learning framework for cryptocurrency portfolio management, employing a deterministic policy gradient algorithm to optimize a portfolio of multiple cryptocurrencies. Their approach demonstrated superior performance compared to traditional portfolio optimization techniques, highlighting the potential of DRL in managing diverse asset portfolios.

Building upon the DQN algorithm, van Hasselt et al. [18] introduced the Double Deep Q-Network (DDQN) to address the issue of overestimation bias in Q-learning. By decoupling the selection and evaluation of actions, DDQN provides more stable and accurate value estimates, leading to improved performance in various domains. This advancement has spurred the adoption of DDQN in financial trading applications.

Li et al. [9] conducted a comprehensive empirical study comparing various DRL algorithms, including DQN, DDQN, and Dueling DQN, for stock trading. Their findings suggest that DRL agents can outperform traditional technical analysis-based strategies, particularly in volatile market conditions. However, they also highlight the sensitivity of DRL performance to hyperparameter settings and the importance of robust training procedures.

While the majority of DRL research in finance has focused on stock trading, the unique characteristics of cryptocurrency markets, such as high volatility and 24/7 trading, present both opportunities and challenges for DRL agents. Zhang et al. [3] analyzed the relationships between returns and liquidity in cryptocurrency markets, demonstrating the complex inter- and intra-asset dependencies that can be exploited by sophisticated trading strategies.

Despite the promising results, several challenges remain in the application of DRL to financial trading. As highlighted by Dulac-Arnold et al. [11], real-world financial environments are characterized by non-stationarity, partial observability, and high-dimensional continuous state and action spaces. These factors necessitate the development of robust and adaptable DRL algorithms capable of handling the complexities of financial markets.

3 Data

This study utilizes historical cryptocurrency price data for Bitcoin (BTC) obtained from Yahoo Finance [10]. The raw data includes open-high-low-close volumes (OHLCV), from which various technical indicators are derived to enhance the trading strategy. A key parameter in calculating these technical indicators is the window size, which represents the number of past periods considered for each calculation, allowing the indicators to capture market patterns over consistent timeframes. Below, we explain each indicator calculated and incorporated into the feature set for our models.

Price

- **Price:** Represents the closing price for the asset, serving as the reference point for other technical calculations.

Momentum Indicators

- **Relative Strength Index (RSI):** Calculates the velocity and magnitude of directional price movements. It is computed as:

$$RSI = 100 - \frac{100}{1 + RS}$$

where RS is the average gain of up periods divided by the average loss of down periods over the past window periods.

- **Momentum (MOM):** Measures the rate of change in the asset's price over window periods:

$$MOM = P_t - P_{t-window}$$

- **Stochastic Oscillator %K:** Measures the current price relative to its range over the past window periods:

$$\%K = \frac{(P_t - L_{window})}{(H_{window} - L_{window})} \times 100$$

where L_{window} and H_{window} are the lowest and highest prices in the past window periods.

- **Stochastic Oscillator %D**: The 3-period moving average of %K:

$$\%D = \frac{\sum_{i=0}^2 \%K_{t-i}}{3}$$

- **Money Flow Index (MFI)**: Incorporates both price and volume to identify overbought or oversold conditions over window periods:

$$MFI = 100 - \frac{100}{1 + \text{Money Ratio}}$$

$$\text{Money Ratio} = \frac{\text{Positive Money Flow}}{\text{Negative Money Flow}}$$

- **True Strength Index (TSI)**: A double smoothed measure of price momentum:

$$TSI = 100 \times \frac{\text{EMA}(\text{EMA}(P_t - P_{t-1}, 13), 7)}{\text{EMA}(\text{EMA}(|P_t - P_{t-1}|, 13), 7)}$$

- **Kaufman's Adaptive Moving Average (KAMA)**: A volatility-adjusted moving average:

$$KAMA = \text{Previous KAMA} + SC \times (P_t - \text{Previous KAMA})$$

$$SC = (\text{ER} \times (2/(2+1) - 2/(30+1)) + 2/(30+1))^2$$

$$\text{ER} = \frac{\text{Change}}{\text{Volatility}}$$

Trend Indicators

- **Simple Moving Average (SMA)**: The average of the asset's closing prices over the last window periods:

$$SMA_{window} = \frac{\sum_{i=0}^{19} P_{t-i}}{window}$$

- **Exponential Moving Average (EMA)**: Places more weight on the most recent prices:

$$EMA_{window} = (P_t \times \frac{2}{window + 1}) + EMA_{\text{yesterday}} \times (1 - \frac{2}{window + 1})$$

- **Moving Average Convergence Divergence (MACD)**: Shows the relationship between two moving averages of prices:

$$MACD = EMA_{12}(P_t) - EMA_{26}(P_t)$$

$$\text{Signal Line} = EMA_9(MACD)$$

- **Trix (TRIX)**: Shows the percent rate of change of a triple exponentially smoothed moving average:

$$TRIX = \frac{EMA_{\text{today}} - EMA_{\text{yesterday}}}{EMA_{\text{yesterday}}}$$

- **Mass Index (MI)**: Uses the high-low range to identify reversals by measuring the expansion or contraction:

$$MI = \sum_{i=1}^{window} \frac{EMA(\text{High} - \text{Low}, 9)}{EMA(EMA(\text{High} - \text{Low}, 9), 9)}$$

- **Commodity Channel Index (CCI)**: Indicates when an instrument moves into an overbought or oversold condition:

$$CCI = \frac{(P_t - SMA_{window}(P_t))}{0.015 \times \text{Mean Deviation}}$$

- **Detrended Price Oscillator (DPO)**: Eliminates long-term trends to focus on short-term patterns:

$$DPO = P_t - SMA_{window}(P_{t-window})$$

- **KST Oscillator (KST)**: Summarizes multiple time-span ROCs into a single momentum oscillator:

$$KST = ROC_{window} \times 1 + ROC_{15} \times 2 + ROC_{window} \times 3 + ROC_{30} \times 4$$

- **Ichimoku Kinkō Hyō (Ichimoku)**: Provides a comprehensive overview by calculating multiple averages:

$$\text{Conversion Line} = \frac{\text{High}_9 + \text{Low}_9}{2}$$

$$\text{Base Line} = \frac{\text{High}_{26} + \text{Low}_{26}}{2}$$

$$\text{Leading Span A} = \frac{\text{Conversion Line} + \text{Base Line}}{2}$$

$$\text{Leading Span B} = \frac{\text{High}_{52} + \text{Low}_{52}}{2}$$

- **Parabolic Stop And Reverse (Parabolic SAR):** Calculates the potential reversals in the market price direction:

$$SAR_{\text{today}} = SAR_{\text{yesterday}} + AF \times (EP - SAR_{\text{yesterday}})$$

where AF (Acceleration Factor) increases by 0.02 each time the extreme point makes a new high.

Volatility Indicators

- **Average True Range (ATR):** Measures market volatility by decomposing the entire range of asset price for that period:

$$ATR = \frac{\sum_{i=1}^{window} \text{True Range}_i}{window}$$

- **Bollinger Bands:** Measures market volatility and assesses if prices are high or low on a relative basis:

$$BB_{\text{upper}} = SMA_{window} + 2 \times \text{StdDev}_{window}$$

$$BB_{\text{lower}} = SMA_{window} - 2 \times \text{StdDev}_{window}$$

Volume Indicators

- **On Balance Volume (OBV):** Measures cumulative buying and selling pressure by adding the volume on up days and subtracting on down days:

$$OBV = \text{previous OBV} + (\text{if closing price } > \text{ previous closing price} \times \text{volume, else if closing price} < \text{ previous closing price} \times (-\text{volume}))$$

- **Chaikin Money Flow (CMF):** Combines price and volume to measure the buying and selling pressure over a set period, typically window periods:

$$CMF = \frac{\sum_{i=1}^{window} \left(\frac{(C_i - L_i) - (H_i - C_i)}{(H_i - L_i)} \times V_i \right)}{\sum_{i=1}^{window} V_i}$$

- **Volume-price Trend (VPT):** Measures the cumulative volume that is added or subtracted from a total starting value depending on the direction of the price move:

$$VPT = \text{previous VPT} + \left(\text{volume} \times \frac{\text{price change}}{\text{previous price}} \right)$$

- **Negative Volume Index (NVI):** Focuses on days when the volume decreases from the previous session. If the price increases on such days, NVI is adjusted upwards:

$$NVI = \text{previous NVI} + (\text{if volume} < \text{previous volume and closing price} > \text{previous closing price})$$

These indicators are calculated to provide a deep insight into the price behavior, market dynamics, and help in making informed trading decisions based on technical analysis.

4 Methods

In this section, we provide a detailed explanation of the Double Deep Q-Network (DDQN) methodology applied to algorithmic trading. The method is carefully designed to capture and learn from historical trading data, allowing an agent to make informed decisions based on market conditions. We discuss the structure of the DDQN model, including the essential components such as the state, action, and reward function, each of which plays a critical role in guiding the agent’s behavior. We also describe the additional mechanisms that support stable learning and optimal decision-making.

The implementation of our DDQN approach is based on a GitHub repository by DeepNeuralAI, which we cloned and adapted for our specific use case [17].

4.1 Overview of the Proposed Method: Double Deep Q-Network (DDQN)

The Double Deep Q-Network (DDQN) implementation consists of several interacting components that work together to create an effective trading system, as illustrated in Figure 4.1. At its core, the Environment represents the trading market, providing the agent with state information (Section 4.1.1) and receiving actions (Section 4.1.2) in return. The Q-Network (online network) interfaces directly with the Environment, selecting actions based on the current state and evaluating them using the reward function (Section 4.1.3). These interactions are stored in the Replay Memory (Section 4.1.5), which maintains a history of experiences (state, action, reward, next state tuples). As shown in the diagram, the Replay Memory serves as a central repository that both stores experiences from the Environment and provides training samples to the networks. The Target-Network, discussed in detail in Section 4.1.6, is periodically updated with the Q-Network’s weights every N intervals to provide stable Q-value estimates. This periodic synchronization, represented by the "Copies every N intervals" connection in Figure 4.1, is crucial for maintaining stable learning targets.

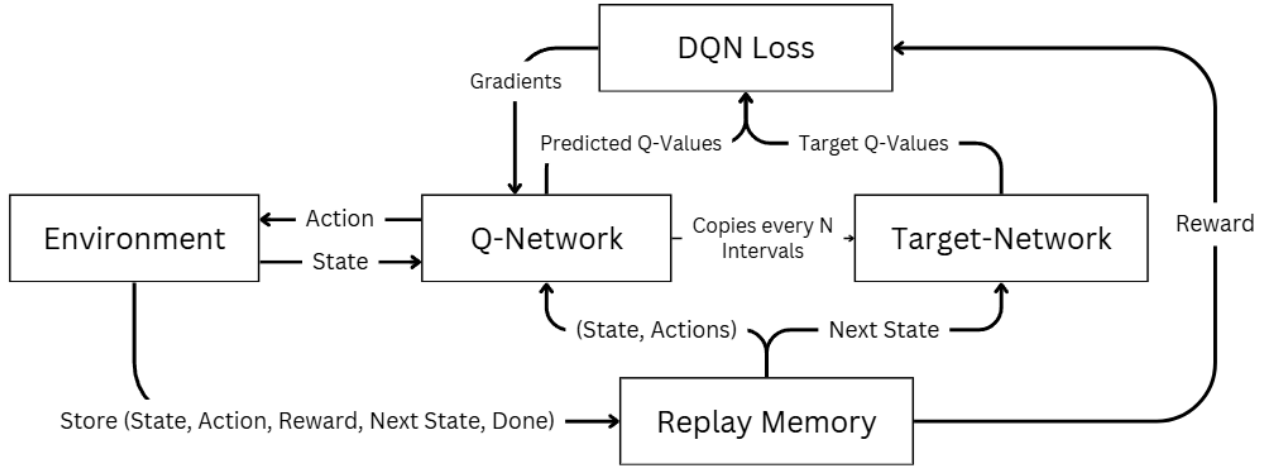


Figure 4.1: DDQN Model Structure

4.1.1 State

The state s_t at each time step t is represented by a vector comprising the past n time steps of normalized technical indicator values, including the closing price. Formally:

$$s_t = [v_{t-n+1}, v_{t-n+2}, \dots, v_t]$$

where v_i is a vector of normalized technical indicator values at time step i , including the normalized closing price. The normalization is performed using min-max scaling:

$$v_{i,\text{norm}} = \frac{v_i - \min(v)}{\max(v) - \min(v)}$$

where the min and max values are calculated over the entire training dataset for each indicator.

4.1.2 Action

The *action* is the decision that the agent makes based on the observed state. The action space consists of three discrete actions:

- **Buy** ($a_t = 1$): Increase the current long position or decrease the current short position.

- **Sell** ($a_t = 2$): Decrease the current long position or increase the current short position.
- **Hold** ($a_t = 0$): Maintain the current position.

Long-Only Strategy

In the long-only strategy, the agent can only hold a positive or zero position. The maximum position size is limited to 100 units, and actions are taken in increments of 100 units. The position update rules are as follows:

- If $a_t = 1$ (Buy) and the current position p_t is less than 100, the position is increased by 100 units, up to a maximum of 100.
- If $a_t = 2$ (Sell) and the current position p_t is greater than 0, the position is decreased by 100 units, down to a minimum of 0.
- If $a_t = 0$ (Hold), the position remains unchanged.

Formally, the position update for long-only can be expressed as:

$$p_{t+1} = \begin{cases} \min(p_t + 100, 100) & \text{if } a_t = 1 \text{ and } p_t < 100 \\ \max(p_t - 100, 0) & \text{if } a_t = 2 \text{ and } p_t > 0 \\ p_t & \text{if } a_t = 0 \end{cases}$$

where p_t is the position at time t .

Short-Selling Strategy

In the short-selling strategy, the agent can hold both positive (long) and negative (short) positions. The position sizes are restricted to increments of 100 units, with a maximum long position of 100 and a maximum short position of -100. The position update rules are as follows:

- If $a_t = 1$ (Buy) and the current position is 0, the position is increased to 100.
- If $a_t = 1$ (Buy) and the current position is -100, the position is increased to 100 (closing the short position and opening a long position).
- If $a_t = 2$ (Sell) and the current position is 0, the position is decreased to -100.
- If $a_t = 2$ (Sell) and the current position is 100, the position is decreased to -100 (closing the long position and opening a short position).

- If $a_t = 0$ (Hold), the position remains unchanged.

Formally, the position update for short-selling can be expressed as:

$$p_{t+1} = \begin{cases} 100 & \text{if } a_t = 1 \text{ and } p_t = 0 \\ 100 & \text{if } a_t = 1 \text{ and } p_t = -100 \\ -100 & \text{if } a_t = 2 \text{ and } p_t = 0 \\ -100 & \text{if } a_t = 2 \text{ and } p_t = 100 \\ p_t & \text{if } a_t = 0 \end{cases}$$

where p_t is the position at time t .

4.1.3 Reward Function

The reward function is designed to incentivize profitable trading decisions by considering the percentage change in the asset's price and the agent's current holdings. The reward r_t at each time step t is calculated as:

$$r_t = \text{pct_change} \times \text{net_holdings}$$

where:

- **pct_change** is the percentage change in the asset's closing price over a specified window period, calculated as:

$$\text{pct_change} = \frac{p_t - p_{t-\text{window}}}{p_{t-\text{window}}}$$

with p_t being the closing price at time t , and the window is specified by the window size parameter.

- **net_holdings** is the net position of the agent at time t , which is positive for long positions and negative for short positions.

This function encourages the agent to maximize cumulative rewards by making decisions that lead to profitable outcomes, considering both long and short positions.

4.1.4 DQN Loss

The DQN Loss function plays a crucial role in our DDQN implementation as it quantifies the difference between predicted and target Q-values, guiding the network's learning process. In our implementation, we utilize the Huber loss function,

which combines the best properties of Mean Squared Error (MSE) and Mean Absolute Error (MAE). The Huber loss is defined as:

$$L_{\delta}(y, f(x)) = \begin{cases} \frac{1}{2}(y - f(x))^2 & \text{for } |y - f(x)| \leq \delta \\ \delta|y - f(x)| - \frac{1}{2}\delta^2 & \text{otherwise} \end{cases}$$

where y is the target Q-value and $f(x)$ is the predicted Q-value. This loss function is particularly suitable for our trading application because it:

- Is less sensitive to outliers compared to MSE, providing more robust training when faced with sudden market movements
- Combines quadratic behavior near zero with linear behavior in the tails, allowing for both precise updates for small errors and controlled updates for large errors
- Helps prevent exploding gradients, which is especially important in financial markets where price movements can be volatile

The target Q-values are calculated using the DDQN algorithm’s double learning mechanism:

$$y_i = r_i + \gamma Q_{\text{target}}(s_{i+1}, \underset{a}{\operatorname{argmax}} Q_{\text{online}}(s_{i+1}, a))$$

where r_i is the immediate reward, γ is the discount factor set to 0.99, and Q_{target} and Q_{online} are the target and online networks respectively.

4.1.5 Experience Replay

In our implementation, experience replay is managed through a deque data structure with a maximum length defined by the replay memory size. Each experience tuple contains (state, action, reward, next state, done) information, where 'done' indicates whether the episode has ended. During training, the agent samples random batches of size specified by the batch size parameter from this memory buffer for learning, helping to break temporal correlations in the training data and providing more stable learning behavior. The replay mechanism is integrated into the agent’s training loop, where experiences are continuously added to the buffer during interaction with the environment, and batches are regularly sampled during the model update phase. This implementation allows the agent to learn from a diverse set of market conditions and trading scenarios.

4.1.6 Target Network Update

Our DDQN implementation maintains two networks: the online network (Q-Network) for action selection and the target network for action evaluation. The target network’s weights are synchronized with the online network every N iterations. This periodic update schedule helps stabilize training by providing a fixed target for Q-value estimation over multiple training steps. The target network is created as a clone of the main network at initialization and is used specifically during the training process to compute target Q-values, reducing overestimation bias and providing more stable learning targets.

4.1.7 Exploration-Exploitation Balance

The exploration-exploitation trade-off is implemented using an epsilon-greedy strategy with decay. The initial exploration rate ($\epsilon_{\min}$), which gradually decays by the exploration decay factor (radr) after each time the agent performs a replay training update (which occurs when the memory buffer contains enough samples), continuing until it reaches a minimum exploration rate ($\epsilon_{\min}$). During evaluation, the exploration is disabled to allow the agent to exploit its learned policy fully. This adaptive exploration strategy allows the agent to extensively explore the action space in the early stages of training while gradually transitioning to a more exploitative strategy as it gains experience in the trading environment. The exploration rate decay is managed within the replay method, ensuring a smooth transition from exploration to exploitation throughout the training process.

4.1.8 Summary of Training Procedure

The training procedure follows these steps:

Algorithm 1 Training Procedure for DDQN Agent

- 1: **Input:**
 - $\mathcal{B}$ – Replay buffer
 - θ – Initial network parameters
 - θ^- – Copy of θ
 - N_r – Replay buffer maximum size
 - N_b – Training batch size
 - N – Target network replacement frequency
 - α – Learning rate
 - 2: Initialize the agent with random weights in both the online network θ and the target network θ^- .
 - 3: **for** each episode **do**
 - 4: Observe the current state s_t .
 - 5: Select action a_t (buy, sell, hold) using the ϵ -greedy policy.
 - 6: Execute the action and observe the reward r_t and next state s_{t+1} .
 - 7: Add transition tuple (s_t, a_t, r_t, s_{t+1}) to replay buffer $\mathcal{B}$, replacing the oldest experience if $|\mathcal{B}| > N_r$.
 - 8: Sample a mini-batch of N_b transitions (s, a, r, s') from $\mathcal{B}$.
 - 9: Calculate target values for each tuple in the mini-batch:

$$y = \begin{cases} r & \text{if } s' \text{ is terminal;} \\ r + \gamma \cdot Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-) & \text{otherwise.} \end{cases}$$
 - 10: Update the online network by performing a gradient descent step to minimize the loss $|y - Q(s, a; \theta)|^2$.
 - 11: **if** every N steps **then**
 - 12: Update target network parameters $\theta^- \leftarrow \theta$.
 - 13: **end if**
 - 14: **end for**
 - 15: **End of episode:** Record the profit and evaluate the model's performance.
-

Summary: The algorithm outlines an iterative learning process where the DDQN agent interacts with the trading environment while continuously improving its decision-making capabilities. The key mechanism for learning from past experiences is the replay buffer $\mathcal{B}$, which stores up to 10,000 transition tuples (state, action, reward, next state). During each training step, the agent samples random batches of 32 transitions from this buffer to update its Q-values. The performance

of previous episodes does not directly influence the next episode's decisions. Instead, the influence is indirect through the stored experiences in the replay buffer, which are used to update the Q-network's parameters θ . These updates gradually improve the agent's ability to estimate action values, affecting future decision-making. The random sampling from the replay buffer ensures that the agent learns from a diverse set of experiences rather than being biased by recent performance, promoting more stable and robust learning behavior.

5 Results

This chapter presents an analysis of the Double Deep Q-Network (DDQN)-based trading agent’s performance under varying market conditions and strategy configurations. The study begins with the introduction of a long-only approach, which restricts the actor to returns resulting exclusively from price increases. Subsequently, a short-selling variant that allows the agent to potentially profit from both price increases and decreases.

To evaluate the adaptability and robustness of the learned policies, three distinct training/validation splits of Bitcoin (BTC) price data are employed. In each scenario, the agent is trained on historical data up to a specified cutoff date (shown in blue in Figure 5.1), followed by a validation period (shown in green) that tests the policy’s performance on out-of-sample market conditions. By varying the cutoff points, different levels of historical context are provided, thereby allowing an assessment of how additional information influences the strategy’s ability to generalize and navigate unforeseen price regimes.

Figure 5.1 illustrates these three scenarios. Each training period ends at a different point, and the subsequent validation segment offers a new, unseen market environment. This approach enables the identification of whether increased historical insight leads to more robust and flexible trading decisions, and whether the strategy maintains effectiveness as market conditions evolve.

The key performance metrics computed to compare the RL-based strategies against the **Buy & Hold baseline** are as follows:

- **Cumulative Return (CR)**: The total return achieved by the strategy over the **entire dataset**, including both the training and validation periods. The model requires a full historical context to make informed decisions during the evaluation phase.
- **Average Daily Returns (ADR)**: Mean of the daily returns, indicating consistency in performance.
- **Standard Deviation of Daily Returns (StdDev)**: A measure of return variability, capturing the risk associated with the strategy.
- **Sharpe Ratio (SR)**: A risk-adjusted measure of performance, reflecting the return achieved per unit of volatility.

By comparing these metrics across the short-selling and long-only as well as against the Buy & Hold reference, insights are gained into the conditions under which the RL-based approach either excels or underperforms. The sections that follow present hyperparameter selections, describe training stability metrics, and then examine each variant’s results in detail. Through this structured analysis, the influence of market regimes, historical data volume, and strategic modifications on the DDQN-based trading agent’s effectiveness is clarified.



Figure 5.1: BTC price trajectories illustrating three distinct training (blue) and validation (green) periods. Each split provides a unique cutoff date that separates historical training data from the subsequent out-of-sample validation segment, enabling an assessment of how additional historical context affects adaptability to new market regimes.

5.1 Hyperparameter Overview

Hyperparameter	Value
Window size	10
Batch size	32
Number of training episodes	50
Learning rate (α)	1×10^{-5}
Discount factor (γ)	0.99
Minimum exploration rate ($\epsilon_{\min}$)	0.01
Exploration decay (radr)	0.995
Replay memory size	10,000
Network architecture	Dense: [256, 512, 512, 256, output=3]
Activation function	ReLU
Loss function	Huber
Optimizer	Adam

Table 5.1: Baseline hyperparameters for the DDQN agent. These parameters were selected as initial values based on general considerations for trading strategies, aiming to establish a stable foundation for experimentation.

Table 5.1 presents the baseline hyperparameters used for the DDQN agent. These parameters were selected as initial values based on general considerations for trading strategies, aiming to provide a starting point for experimentation. The window size of 10 is intended to capture recent market conditions while balancing sensitivity to short-term fluctuations. A batch size of 32 and a replay memory of 10,000 are common choices designed to provide a diverse range of experiences for training, potentially reducing the risk of overfitting to specific market patterns.

The learning rate (1×10^{-5}) and the discount factor ($\gamma = 0.99$) were chosen to potentially support gradual policy improvements while emphasizing long-term returns. A relatively low learning rate is intended to prevent erratic parameter shifts, which may help maintain stability in a volatile market environment. Similarly, a high discount factor aims to encourage the agent to prioritize future reward signals rather than solely focusing on immediate gains.

Exploration parameters were set to influence how the agent adapts to price uncertainty. A minimum exploration rate of 0.01 is expected to ensure that even after prolonged training, the agent retains the ability to explore less familiar actions, which might prove beneficial during sudden market shifts. The chosen exploration decay (0.995) is designed to steadily reduce random behavior, potentially allowing the agent to refine its policy as it acquires more knowledge. The Huber loss and Adam optimizer were selected to promote stable gradient updates. The network

architecture, consisting of fully connected dense layers with 256 and 512 units, was chosen for its simplicity and computational efficiency. While this design is straightforward and expected to effectively process the price data and technical indicators, more complex architectures could potentially enhance performance but were not explored in this study.

5.2 Training Performance

This section provides an overview of the training process using a representative run from the set of conducted experiments. Although individual episodes and final results vary slightly across the different configurations and training/validation splits, the overall learning dynamics remain qualitatively similar, ensuring that the presented curves offer a reliable indication of the general training behavior.

Figure 5.2 displays the cumulative rewards achieved by the agent at the end of each training episode. Initially, there is a rapid increase in cumulative returns, followed by a plateau as the agent refines its policy. After approximately 10 to 15 episodes, the cumulative reward growth rate stabilizes, suggesting that the agent has discovered a reasonably effective strategy. Minor fluctuations beyond this point typically reflect incremental fine-tuning rather than fundamental changes in policy quality.

Figure 5.3 presents the training loss over the course of the same run. The loss decreases sharply during the early episodes, indicating that the agent's Q-value estimates are initially imprecise but improve rapidly as more experiences are processed and incorporated. By around the 20th episode, the loss curve becomes considerably flatter, and by the end of the 50-episode training period, the loss values are near a steady minimum. This stable plateau in the loss signals that the model parameters have converged to a point where further improvements are incremental and less pronounced.

In summary, these plots demonstrate that the training process leads to a stable and effective policy within a relatively short span of episodes. The observed patterns are representative of the overall training behavior noted in all tested variants, indicating that the chosen hyperparameters and methodological approach provide a robust foundation for learning a competent trading strategy.

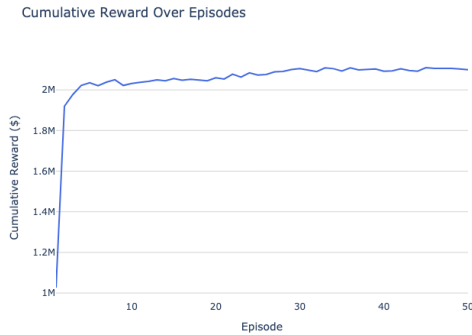


Figure 5.2: Cumulative rewards per episode, showing rapid improvement during the initial phase and stabilization thereafter.



Figure 5.3: Training loss over episodes, demonstrating a decrease as the agent’s Q-value estimates improve and eventual stabilization at low loss levels.

5.3 Long-Only Variant

In this section, the results of the long-only variant of the trading strategy are analyzed. Unlike the short-selling variant, this approach restricts the agent to holding or selling assets without the option of shorting. The agent operates with a maximum position size of 100 units, ensuring controlled trading behavior. The results are evaluated for three different training cutoff dates, defining the boundary between the training set (up to the cutoff) and the validation set (subsequent period). Buy and sell signals are plotted against the BTC price trajectory, and the performance is benchmarked against the Buy & Hold strategy.

Figure 5.4 shows the results for the validation period starting from September 2020, after the agent has been trained on data up to August 2020. During this phase, the BTC market transitions into a strong bull run. However, the agent, having not encountered such rapid upward trends during its training, struggles to fully capitalize on the rally. While it identifies early opportunities and executes several profitable trades during the initial price increase, its premature sell actions result in missed gains during the sustained upward rallies. This cautious behavior, aimed at avoiding minor corrections, limits short-term losses but prevents the agent from fully exploiting the bull market. In contrast, the Buy & Hold strategy benefits from full exposure to the rally, achieving significantly higher cumulative returns. Despite these limitations, the agent achieves a Sharpe Ratio of 0.8559,

indicating smoother performance with reduced volatility compared to the Buy & Hold approach.

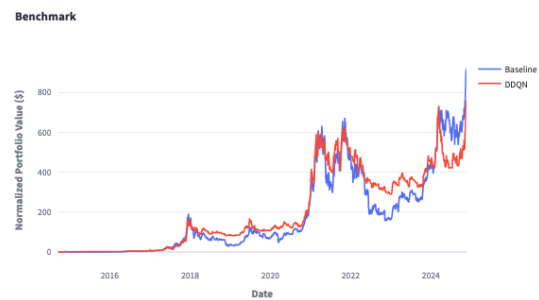
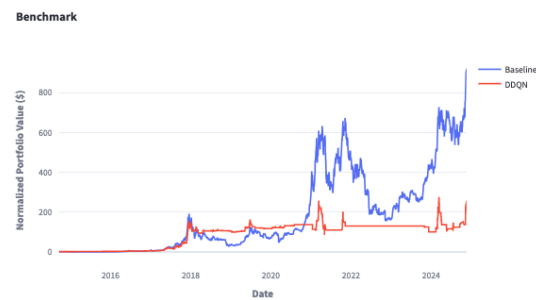
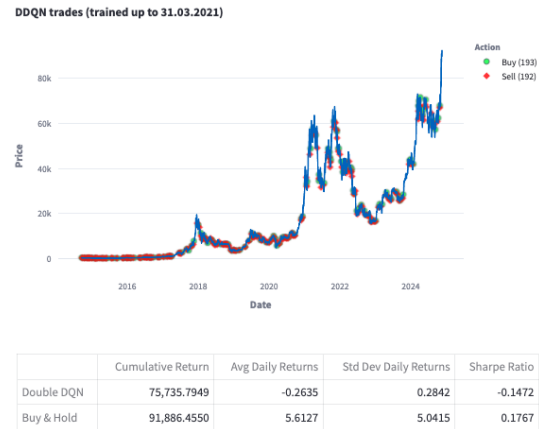
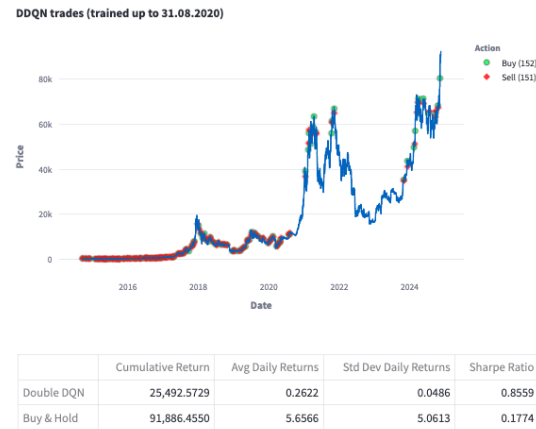


Figure 5.4: Long-only variant results for the training cutoff at 31.08.2020.

Figure 5.5: Long-only variant results for the training cutoff at 31.03.2021.

Figure 5.5 presents results for the validation period starting from April 2021, after the agent has been trained on data up to March 2021. This phase includes Bitcoin's correction following its all-time highs in early 2021. The long-only agent effectively mitigates sharp declines during market downturns in May 2021 and May 2022 by exiting positions, reducing significant losses. However, the agent struggles during March 2024, as it fails to adapt to rapid price drops, resulting in notable losses. This cautious yet inconsistent approach contrasts with the Buy & Hold strategy, which benefits from continuous exposure and achieves higher cumulative returns during recovery phases. The agent's Sharpe Ratio of -0.1472 highlights the challenges of maintaining consistent performance in volatile markets, especially during periods of sudden trend reversals.

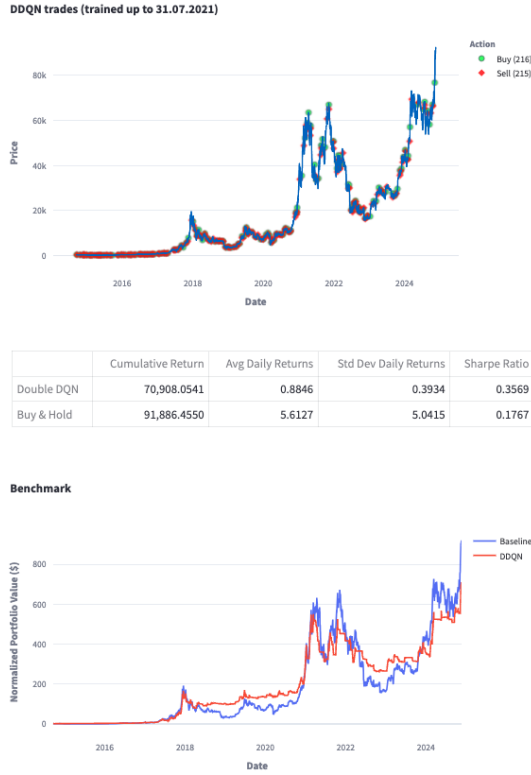


Figure 5.6: Long-only variant results for the training cutoff at 31.07.2021.

Figure 5.6 examines the validation period starting from August 2021, which begins approximately between the two major peaks of the 2021-2022 cycle. During this period, the agent demonstrates improved risk management by successfully mitigating losses during sharp downturns. Notably, in late 2021 and early 2022, the agent effectively exits positions ahead of steep corrections, preserving capital during the declining phases. However, the agent struggles to re-enter the market promptly during subsequent recoveries, leading to missed opportunities for gains during rebounds. This behavior underscores the agent’s cautious trading strategy, which prioritizes avoiding losses over aggressively pursuing profits. Despite these limitations, the agent achieves a Sharpe Ratio of 0.3569, reflecting enhanced risk-adjusted performance compared to earlier phases. The improved Sharpe Ratio highlights the agent’s ability to manage volatility more effectively in this challenging market environment.

Figure 5.7 highlights specific trade during June-July 2023. The agent reacts to a sharp upward movement with a delayed buy action, missing the initial price

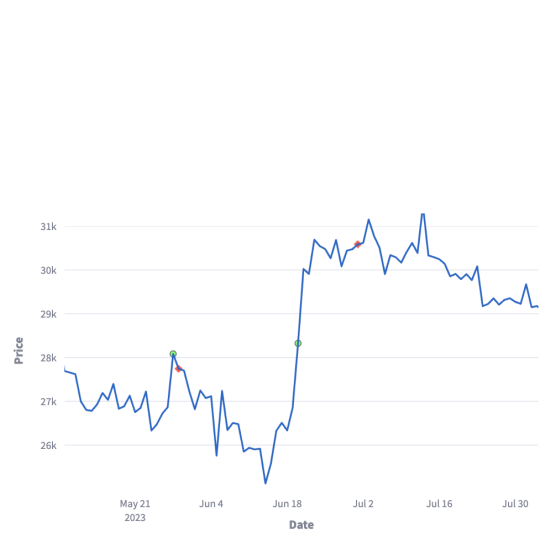


Figure 5.7: Specific trade during June-July 2023, showing delayed buy action in response to market volatility.

increase. This reflects the challenges of adapting to sudden and volatile market changes. While the trade eventually aligns with the upward trend, the delay results in suboptimal performance compared to continuous exposure under Buy & Hold.

Overall, the long-only variant shows improved performance in managing risk during corrections, particularly in environments with downward or sideways price movements. However, during strong bull runs, such as those observed in early 2021, the frequent sell actions result in missed gains compared to the Buy & Hold strategy. The Sharpe Ratio values indicate that the agent achieves smoother returns with reduced volatility, particularly in phases where the market experiences moderate trends or corrections. The long-only agent demonstrates its effectiveness in avoiding short-term losses but struggles to fully capitalize on sharp upward price movements, particularly when delays in re-entry occur.

5.4 Short-Selling Variant

In this section, the results of the short-selling variant of the trading strategy are analyzed. This variant allows the agent to profit from both upward and downward price movements. The agent can borrow assets and sell them, intending to repurchase them at a lower price later. Position sizes are restricted to increments of 100 units to ensure realistic trading conditions and prevent the accumulation of excessively large holdings.

Figures 5.8, 5.10, and 5.11 illustrate the trading behavior of the agent under three different training cutoff dates. These cutoff dates define the boundary between the training set (up to the cutoff) and the validation set (subsequent period). Buy and sell signals are plotted against the BTC price trajectory, highlighting where short positions were opened to exploit perceived market declines. The results are evaluated in detail, focusing on the impact of the short-selling strategy compared to the Buy & Hold benchmark and analyzing how market conditions influence the agent's performance.

Figure 5.8 shows the results for the period ending 31.08.2020. This period precedes the significant bull run starting in late 2020. The agent attempts to short during temporary corrections; however, certain trades occur just before the market resumes a strong upward trend. For instance, a short position opened on 2020-08-11 (highlighted in Figure 5.9) results in significant losses as the market begins its rally. In a bullish environment, such mistimed short positions can substantially harm performance, as the agent misses the upward momentum captured by the Buy & Hold strategy.

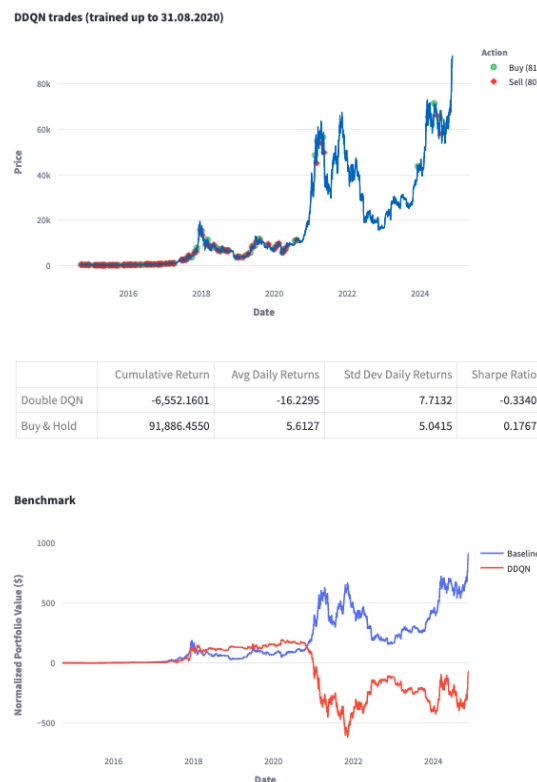


Figure 5.8: Short-selling variant results for the training cutoff at 31.08.2020.



Figure 5.9: A specific short trade initiated on 2020-08-11. The agent opened a short position before a significant price rally, leading to losses.

Figure 5.10 presents results for the validation period starting from April 2021. In this short-selling variant, the agent is familiar with data up to the first major peak but encounters unseen market conditions thereafter. During the validation period, the BTC market experienced rapid growth, peaking in late 2021, followed by sharp corrections in early 2022, and eventually a significant rebound in 2024.

The agent demonstrates effectiveness in capturing some downward movements during corrections, particularly in early 2022. However, frequent mistimed short positions in strongly bullish phases, such as mid to late 2021, significantly hinder performance. This behavior reflects the difficulty of short-selling during sustained uptrends, where short positions are repeatedly invalidated by strong upward momentum.

Additionally, the agent fails to identify the major rebound in 2024, missing an opportunity to adjust its policy in response to the recovering market. This inability to adapt to sharp recoveries highlights a critical limitation of the short-selling strategy in volatile and predominantly bullish markets. The underperformance

relative to the Buy & Hold strategy underscores the inherent challenges of relying on short positions in markets characterized by long-term upward trends and sporadic corrections.

Figure 5.11 highlights results for the validation period starting August 2021. Compared to the 31.03.2021 variant, the agent performs better during downward movements in late 2021 and early 2022, demonstrating improved timing and risk management. While the rebound in 2024 is still not fully captured, the agent avoids the significant losses seen previously, reflecting better adaptation to volatile conditions with the extended training period.

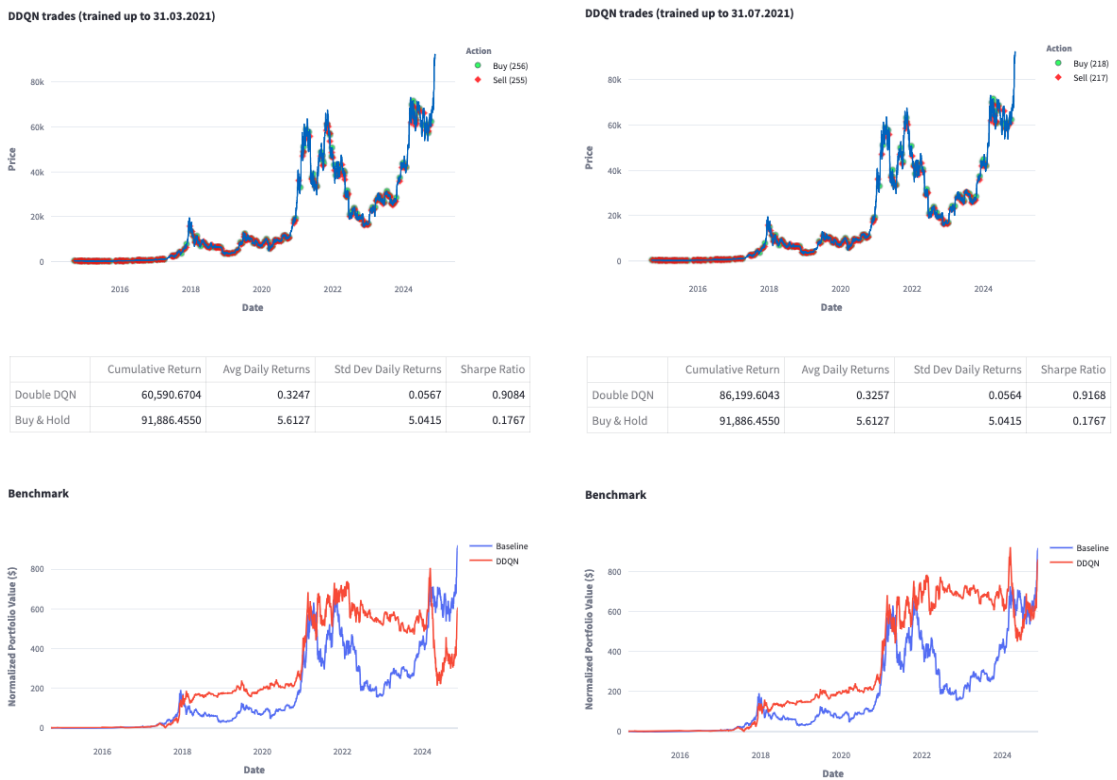


Figure 5.10: Short-selling variant results for the training cutoff at 31.03.2021.

Figure 5.11: Short-selling variant results for the training cutoff at 31.07.2021.

Overall, the short-selling variant introduces greater variability and risk, especially in bullish markets, where mistimed shorts result in significant losses. In contrast, the long-only variant demonstrates more consistent performance across the three different training configurations, benefiting from its alignment with Bitcoin’s predominantly bullish trends. The short-selling variant shows improvements

during downward or non-trending markets but remains less reliable compared to the long-only approach.

5.5 Tuned Long-Only Variant

Following the analysis of the baseline Long-Only variant (Figure 5.6), this section introduces a tuned version of the Long-Only strategy, where key hyperparameters were manually adjusted to assess potential improvements in performance. The changes include a reduced window size of 5 (compared to the baseline’s 10), a higher learning rate of 1×10^{-3} , a discount factor of 0.90, and a reduced exploration decay rate of 0.985. These adjustments aim to test whether shorter time horizons and quicker policy updates enhance the agent’s responsiveness to rapid market fluctuations.

The tuned parameters are summarized as follows:

Hyperparameter	Value
Window size	5
Batch size	32
Number of training episodes	50
Learning rate (α)	1×10^{-3}
Discount factor (γ)	0.90
Minimum exploration rate ($\epsilon_{\min}$)	0.01
Exploration decay (radr)	0.985
Replay memory size	10,000
Network architecture	Dense: [256, 512, 512, 256, output=3]
Activation function	ReLU
Loss function	Huber
Optimizer	Adam

Table 5.2: Hyperparameters for the tuned DDQN agent. These adjustments are designed to enhance the agent’s ability to adapt to rapid market changes and optimize decision-making in volatile conditions.

5.5.1 Performance Comparison

The tuned variant was evaluated on the same training and validation splits as the baseline Long-Only agent. Figure 5.12 presents the results of the tuned agent for the cutoff date of 31.07.2021, with the corresponding performance metrics listed alongside those of the baseline agent in Table 5.3.

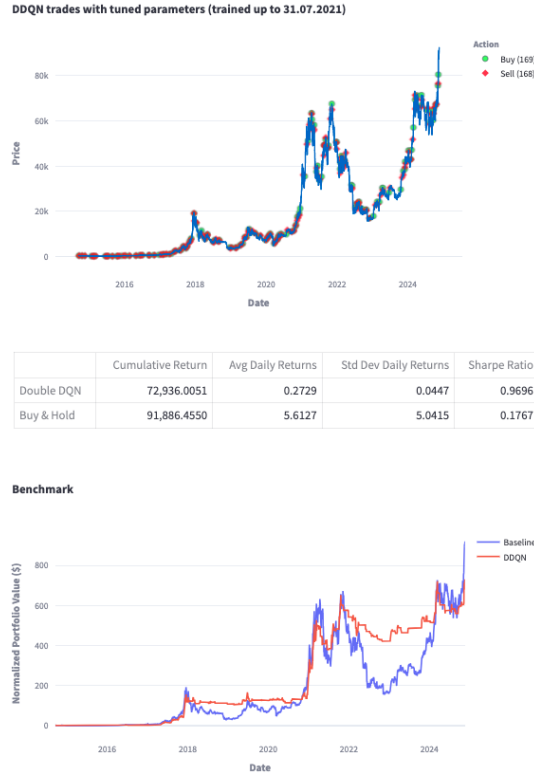


Figure 5.12: Tuned Long-Only variant results for the training cutoff at 31.07.2021.

Variant	Cumulative Return	Avg Daily Returns	Std Dev Daily Returns	Sharpe Ratio
Baseline Long-Only (Figure 5.6)	70,806.1871	0.0846	0.2934	0.2433
Tuned Long-Only (Figure 5.12)	72,936.0051	0.2729	0.0447	0.9696
Buy & Hold	91,886.4550	5.6127	5.0415	0.1767

Table 5.3: Performance comparison of the baseline and tuned Long-Only variants for the training cutoff at 31.07.2021. Key metrics include cumulative return, average daily returns, standard deviation of daily returns, and Sharpe ratio. The best-performing values in each metric are highlighted.

5.5.2 Analysis of Tuned Performance

The tuned variant exhibits notable improvements across all evaluated metrics compared to the baseline. Specifically:

- **Cumulative Return:** The tuned agent achieves a higher cumulative return (72,936.0051) than the baseline, reflecting its ability to capture a greater proportion of upward price movements while avoiding losses during corrections.

- **Average Daily Returns:** The average daily return increases significantly (from 0.0846 to 0.2729), demonstrating that the tuned agent generates more consistent and profitable trades.
- **Sharpe Ratio:** The most striking improvement is observed in the Sharpe Ratio, which rises from 0.2433 to 0.9696. This indicates that the tuned agent achieves significantly better risk-adjusted performance, likely due to the reduced variability in daily returns (standard deviation decreases from 0.2934 to 0.0447).

Compared to the Buy & Hold strategy, the tuned agent achieves a much higher Sharpe Ratio but falls short in terms of cumulative returns. This suggests that while the tuned strategy excels in managing risk and maintaining consistent performance, it does not fully capitalize on the broader upward momentum of the BTC market.

5.5.3 Concluding Remarks on Tuned Parameters

The manual tuning of hyperparameters demonstrates the potential for significant performance gains in DDQN-based trading strategies. The reduced window size and higher learning rate likely contributed to the agent’s improved responsiveness to rapid market changes, while the lower discount factor emphasizes shorter-term rewards, aligning with Bitcoin’s highly volatile nature. These findings underscore the importance of hyperparameter optimization in achieving robust and adaptable trading strategies. Future work could systematically explore additional parameter configurations to further enhance performance across diverse market conditions.

5.6 Comparison and Summary

This section provides a comparison of the results from the two trading variants – with and without short-selling – as well as the additional tuned long-only variant. The key metrics include cumulative return, average daily returns, standard deviation of daily returns, and the Sharpe Ratio. These values were calculated for the three different training/validation cutoff dates.

Variant	Cutoff Date	Cumulative Return	Avg Daily Returns	Std Dev Daily Returns	Sharpe Ratio
Buy & Hold		91,886.4550	5.6127	5.0415	0.1767
Long-Only (Baseline)	31.08.2020	25,492.5729	0.2622	0.0486	0.8559
Long-Only (Baseline)	31.03.2021	75,735.7949	-0.2635	0.2842	-0.1472
Long-Only (Baseline)	31.07.2021	70,806.1871	2.3452	1.5300	0.2433
Long-Only (Tuned)	31.07.2021	72,936.0051	0.2729	0.0447	0.9696
Short-Selling	31.08.2020	-6,552.1601	-16.2295	7.7132	-0.3340
Short-Selling	31.03.2021	60,590.6704	0.3247	0.0567	0.9084
Short-Selling	31.07.2021	86,199.6043	0.3257	0.0564	0.9168

Table 5.4: Comparison of Buy & Hold, short-selling, baseline long-only, and tuned long-only variants across all training cutoff dates. The best-performing metric in each category is highlighted. Buy & Hold results are boxed for emphasis.

5.6.1 Summary of Findings

The comparison reveals notable differences between the short-selling, long-only, and tuned long-only variants:

- **Short-Selling Variant:** The short-selling variant demonstrates strong performance during periods of declining or sideward markets, as evidenced in the 31.07.2021 cutoff. It achieved the highest cumulative return and Sharpe Ratio during this period. However, the strategy struggled in bullish markets, such as the period before the 31.08.2020 cutoff, due to poorly timed short positions that resulted in significant losses.
- **Baseline Long-Only Variant:** The baseline long-only variant performed consistently well during bullish trends, such as the 31.08.2020 cutoff. However, its risk-adjusted returns and Sharpe Ratios were lower compared to the short-selling variant in volatile or sideward-moving markets, such as the 31.07.2021 cutoff.
- **Tuned Long-Only Variant:** The tuned long-only variant, evaluated for the 31.07.2021 cutoff, outperformed the baseline in both risk-adjusted performance and return consistency. With a Sharpe Ratio of 0.9696 and a significantly lower standard deviation of daily returns (0.0447), the tuned variant demonstrated improved adaptability to rapid market fluctuations. However, it still lagged behind the short-selling variant in terms of cumulative return during this period.

These results highlight the trade-offs between different strategies. While the short-selling variant excels in risk-adjusted performance under volatile conditions, it is inherently riskier during strong bullish trends. The baseline long-only variant offers consistent performance in bullish markets but is less effective during periods

of high volatility. The tuned long-only variant bridges this gap to some extent, improving stability and risk management without sacrificing profitability in volatile markets.

6 Discussion and Conclusion

6.1 Discussion

In this section, the results of the Long-Only and Short-Selling variants are discussed in greater detail, focusing on their relative performance, the manually tuned variant, and the underlying reasons for their behavior. The chosen methods, their limitations, and opportunities for future improvement are analyzed.

6.1.1 Long-Only Variant: Interpretation of Results

The Long-Only variant exhibited a more stable and consistent performance compared to the Short-Selling variant, primarily because it avoided the added risks associated with short-selling. However, its performance across the three validation periods reveals both its strengths and limitations under varying market conditions.

For the validation period following the **31.08.2020** cutoff (Figure 5.4), the agent did not fully capitalize on the bull run, as it had never encountered such rapid upward trends in its training data. Despite missing significant gains, the agent avoided large losses by refraining from shorting, maintaining stable returns. This cautious approach demonstrates its ability to mitigate risks even in unfamiliar bullish conditions.

During the **31.03.2021** validation period (Figure 5.5), the agent performed moderately well during the correction phase after the bull run. It effectively exited positions during downward movements, mitigating significant losses. However, it struggled with the rebound in 2024, as delayed re-entries limited its ability to capture the full extent of the recovery. This highlights the agent’s difficulty in adapting quickly to sharp market recoveries.

In the **31.07.2021** validation period (Figure 5.6), the agent showed slightly better performance compared to the 31.03.2021 configuration. Having been trained on data that included the brief correction phase in 2021, the agent demonstrated improved risk management during similar scenarios. It exited positions effectively during downturns and handled rebounds with slightly more precision, though delayed buy actions still limited its ability to fully exploit recoveries.

Overall, the Long-Only variant’s stable and cautious approach resulted in consistent performance across the different market phases. Its avoidance of high-risk

strategie with short-selling, contributed to its stability, while exposure to more diverse training scenarios slightly enhanced its adaptability over time.

6.1.2 Tuned Long-Only Variant

To explore the potential of further improvements, a manually tuned Long-Only variant was tested with adjusted parameters, such as a reduced window size of 5 and a higher learning rate of 1×10^{-3} . As shown in Figure 5.12, the tuned agent exhibited enhanced performance during the **31.07.2021** validation period, achieving a Sharpe Ratio of **0.9696**, compared to **0.2433** in the baseline configuration.

The tuned variant’s lower standard deviation of daily returns (0.0447) highlights its improved ability to manage volatility, while its cumulative return of 72,936.0051 still fell short of the Buy & Hold benchmark. These results suggest that fine-tuning parameters can significantly enhance the agent’s risk-adjusted performance, although there remains room for further exploration and optimization.

6.1.3 Short-Selling Variant: Interpretation of Results

The Short-Selling variant exhibited significant vulnerabilities, particularly during bullish markets, due to its reliance on downward price movements. This made the agent highly susceptible to mistimed trades and large losses, especially in unfamiliar market conditions.

For the **31.08.2020** validation period (Figure 5.8), the agent suffered substantial losses as it failed to recognize the beginning of a major bull run. Poorly timed short positions, such as those in mid-August 2020, coincided with a rapid upward rally, emphasizing the high risks associated with shorting in volatile and bullish markets.

During the **31.03.2021** validation period (Figure 5.10), the agent showed improved performance by effectively capturing profits during the downward corrections in 2022. However, its inability to adapt to subsequent recoveries, including the rebound in 2024, resulted in missed opportunities and moderate losses. Although the agent was slightly more effective than the Long-Only variant during the rebound, the overall performance remained inconsistent.

In the **31.07.2021** validation period (Figure 5.11), the agent demonstrated further improvement, leveraging its experience from prior downward phases to achieve better results during corrections. It also performed slightly better than the Long-Only variant in handling the rebound in 2024. However, the inherent unpredictability of market trends and the sharp rebounds in July 2021 limited its overall effectiveness and underscored the risks of short-selling in volatile environments.

Overall, while the Short-Selling variant became more effective with exposure to additional market scenarios, the high-risk nature of shorting in unpredictable markets poses a significant challenge. The strategy remains particularly vulnerable to large errors, making it far less stable than the Long-Only approach.

6.1.4 Discussion of Methods and Limitations

The results highlight several key insights and areas for refinement in the chosen methods and parameters:

- **Fixed Window Size:** The fixed window size of 10 (baseline) and 5 (tuned) offered different trade-offs. While a smaller window size improved responsiveness in the tuned variant, further experimentation with adaptive window sizes could provide a more optimal balance between short-term reactivity and long-term trend recognition.
- **Learning Rate and Discount Factor:** The baseline learning rate ($\alpha = 1 \times 10^{-5}$) ensured stability but slowed convergence, while the tuned learning rate ($\alpha = 1 \times 10^{-3}$) improved performance during volatile conditions. The discount factor ($\gamma = 0.99$ in baseline and 0.90 in tuned) demonstrated the impact of emphasizing long-term vs. short-term rewards, with the lower γ in the tuned variant enabling a stronger focus on immediate market trends.
- **Network Architecture:** The fully connected dense layers ($[256, 512, 512, 256]$) were chosen for simplicity and computational efficiency. However, alternative architectures designed to handle volatile and dynamic patterns in time-series data could better capture the complexities of Bitcoin’s price movements.
- **Reward Function Design:** The current reward function prioritizes immediate returns, which can lead to excessive trading. Alternative reward structures that penalize frequent trades or incorporate risk-adjusted metrics could align the agent’s actions more closely with long-term profitability.
- **Technical Indicators:** The agent leverages commonly used indicators, but more sophisticated indicators tailored to identifying volatile market patterns could enhance the agent’s ability to adapt to sudden price changes and improve decision-making.

The introduction of the tuned Long-Only variant underscores the potential for parameter optimization to significantly enhance performance. However, further systematic exploration of hyperparameters and architectures is essential to fully realize the capabilities of reinforcement learning in volatile markets like Bitcoin.

6.1.5 The Role of Optimized Data Sources

To address these challenges, integrating carefully chosen and optimized data sources could enhance the agent’s decision-making capabilities. The focus should remain on features that have **no delayed impact** on price data to ensure timely and actionable signals. Specifically:

- **Blockchain Metrics:** Real-time blockchain-specific data, such as transaction fees, hashrate, and block saturation, provide insights into on-chain activity [13]. For example:
 - **Hashrate:** A rising hashrate may signal miner confidence and increased network security, correlating with upward price movements.
 - **Transaction Fees:** Rising fees can indicate network congestion, often preceding significant price action.
 - **Block Saturation:** Higher block utilization reflects increased transaction demand, which can influence market sentiment.

These metrics offer additional predictive value and align closely with Bitcoin price movements, providing the agent with real-time actionable signals.

- **Technical Indicators:** Optimizing the selection of technical indicators is equally crucial. Indicators such as Moving Average Convergence Divergence (MACD) [14], Volume Weighted Average Price (VWAP) [15], and Average True Range (ATR) [16] can provide a broader view of market conditions and help the agent identify trend direction, volatility, and momentum more effectively.

By focusing on real-time blockchain data and refined technical indicators, the agent can gain a deeper understanding of market patterns and adapt more effectively to changing conditions.

6.1.6 Future Work: Algorithmic Improvements

Future research should explore alternative reinforcement learning algorithms that address the limitations of the current Double Deep Q-Network (DDQN) approach. Promising methods include:

- **Advantage Actor-Critic (A2C):** A2C provides a balance between policy optimization and value estimation, improving stability in environments with high volatility [1].

- **Proximal Policy Optimization (PPO):** PPO offers robustness and efficiency by constraining policy updates, which reduces the risk of instability during training [19].
- **Soft Actor-Critic (SAC):** SAC incorporates entropy regularization to encourage exploration, enabling the agent to identify profitable opportunities in highly dynamic markets [20].

These algorithms have demonstrated success in similar reinforcement learning tasks and could offer improvements in stability, adaptability, and overall performance.

6.2 Conclusion

This study highlights the challenges and opportunities of applying reinforcement learning to the volatile Bitcoin market. The Long-Only variant, while less risky, does not consistently outperform the Short-Selling variant. However, short-selling remains significantly riskier, as mistimed decisions in volatile conditions can result in irrecoverable losses.

The current hyperparameters, such as the discount factor γ and fixed window size, provide a foundation for stable training but limit adaptability to sudden market changes. Both the agent and the reward function offer considerable potential for refinement, including more extensive tuning to better align with diverse market dynamics.

Incorporating optimized technical indicators and real-time blockchain metrics could further enhance decision-making, provided they reflect immediate market conditions without delays. Additionally, exploring alternative algorithms and architectures could improve the agent’s robustness in volatile environments.

In summary, while the Long-Only strategy offers a safer option, short-selling strategies can be effective under certain conditions but require precise timing and careful tuning to mitigate risks. Future work should focus on deeper tuning and refinement to fully exploit the agent’s potential in navigating complex market environments.

Bibliography

- [1] Sutton, R.S. and Barto, A.G., *Reinforcement Learning: An Introduction*, MIT Press, 2018.
- [2] Dulac-Arnold, G., Levine, N., Mankowitz, D.J., Li, J., Paduraru, C., Goyal, S., and Hester, T., *Challenges of real-world reinforcement learning: definitions, benchmarks and analysis*, Machine Learning, 2021.
- [3] Zhang, M., Zhu, B., Li, Z., Jin, S., and Xia, Y., *Relationships among return and liquidity of cryptocurrencies*, Financial Innovation, 2024.
- [4] van Hasselt, H., Guez, A., and Silver, D., *Deep reinforcement learning with double Q-learning*, In Proceedings of the AAAI Conference on Artificial Intelligence, 2016.
- [5] Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., Van Den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., *Mastering the game of Go with deep neural networks and tree search*, Nature, 2016.
- [6] Moody, J., Wu, L., Liao, Y., and Saffell, M., *Performance functions and reinforcement learning for trading systems and portfolios*, Journal of Forecasting, 1998.
- [7] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M., Fidjeland, A.K., Ostrovski, G., and Petersen, S., *Human-level control through deep reinforcement learning*, Nature, 2015.
- [8] Jiang, Z., Liang, J., and He, X., *Cryptocurrency portfolio management with deep reinforcement learning*, In 2017 Intelligent Systems Conference (IntelliSys), 2017.
- [9] Li, Y., Ni, P., and Chang, V., *Deep reinforcement learning for stock trading: An empirical study*, In 2019 International Conference on Data Mining Workshops (ICDMW), 2019.
- [10] Yahoo Finance, *Historical Cryptocurrency Data*, Accessed: 2024-09-10, <https://finance.yahoo.com/cryptocurrencies>

- [11] Dulac-Arnold, G., Levine, N., Mankowitz, D.J., Li, J., Paduraru, C., Goyal, S., and Hester, T., *Challenges of real-world reinforcement learning: definitions, benchmarks and analysis*, Machine Learning, 2021.
- [12] Moody, J., Wu, L., Liao, Y., and Saffell, M., *Performance functions and reinforcement learning for trading systems and portfolios*, Journal of Forecasting, 1998.
- [13] Jiang, Z., Liang, J., and He, X., *Cryptocurrency portfolio management with deep reinforcement learning*, In 2017 Intelligent Systems Conference (IntelliSys), 2017.
- [14] P. T. Chio, *A comparative study of the MACD-based trading strategies: evidence from the US stock market*, arXiv preprint arXiv:2206.12282, 2022. Retrieved from <https://arxiv.org/abs/2206.12282>.
- [15] S. Spiroglou, *Volume Weighted Average Price (VWAP): The Holy Grail for Day Traders*, SSRN Electronic Journal, 2022. Retrieved from https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4631351.
- [16] International Journal of Creative Research Thoughts (IJCRT), *Trading through Technical Analysis with MACD and EMA Indicators*, 2021. Retrieved from <https://ijcrt.org/papers/IJCRT2103689.pdf>.
- [17] DeepNeuralAI, *RL-DeepQLearning-Trading*, GitHub repository, 2024, <https://github.com/DeepNeuralAI/RL-DeepQLearning-Trading>.
- [18] van Hasselt, H., Guez, A., and Silver, D., *Deep reinforcement learning with double Q-learning*, In Proceedings of the AAAI Conference on Artificial Intelligence, 2016.
- [19] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O., *Proximal Policy Optimization Algorithms*, arXiv preprint arXiv:1707.06347, 2017.
- [20] Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S., *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*, In Proceedings of the International Conference on Machine Learning (ICML), 2018.

6.3 Appendix

6.3.1 Artificial Intelligence Declaration

The authors acknowledge that parts of this work were supported by AI tools, including OpenAI’s ChatGPT and Anthropic’s Claude. These tools were used to assist in refining text, generating rephrasing suggestions, drafting sections, identifying bugs in code, and structuring the coding workflow.